

Knowledge Representation & Reasoning in AI

Part 1

Why Rules Fail

Part 2

Semantic Networks

Part 3

Hierarchies

Part 4

Frames

Part 5

Ontologies

Why Do We Need Better Knowledge Representation?

Limitations of Forward & Backward Chaining

Part 1 of 5

1.1 What Are Rule-Based Systems?

Rule-Based AI stores knowledge as:

IF (condition) → THEN (action)

Forward Chaining

Start from known FACTS → keep applying rules → arrive at GOAL

Data-driven. Used in business rule engines and expert systems.

Backward Chaining

Start from a GOAL → work backwards → find what facts prove it

Goal-driven. Used in Prolog and medical diagnosis systems.

CLASSIC EXAMPLE — MYCIN Expert System (Stanford, 1970s)

~600 production rules

- ✓ 65–80% diagnostic accuracy — impressive for its era
- ✓ Used by real doctors at Stanford Medical Center

But it exposed fundamental limits that FORCED the field to evolve into better knowledge representation systems.

1.2 The 5 Fundamental Problems with Rule-Based AI (1/2)

1

Knowledge Acquisition Bottleneck

600 rules = 3 years of expert interviews. 6,000 rules = 30 years.
Every new disease requires manually writing new rules.
There is NO way to scale this.

2

No Structural Knowledge

MYCIN matched patterns but had NO idea what 'bacteria' IS.
It could not answer: 'Is bacteria related to viruses?'
Every fact was isolated — zero connections between concepts.

3

Brittleness with Missing Data

If ONE condition in a rule was missing, the ENTIRE chain BROKE.
Real patients forget symptoms. Real doctors have incomplete records.
No partial reasoning — all or nothing.

1.2 The 5 Fundamental Problems with Rule-Based AI (2/2)

4

No Reuse Across Systems

MYCIN's rules worked ONLY for MYCIN. A botany AI had to start from scratch.
No shared vocabulary. No interoperability.
Every system reinvented the wheel.

5

Combinatorial Explosion

50 diseases × 30 symptoms = 1,500 minimum rules.
With symptom combinations: MILLIONS of rules needed.
Forward chaining generates thousands of useless intermediate facts.

KEY INSIGHT: Rule-based AI fails to SCALE, CONNECT, SHARE, or TOLERATE missing data. We need a fundamentally different way to represent knowledge — one that captures STRUCTURE, not just isolated facts.



CLASS ACTIVITY 1.1: Diagnosing the Limits of Rule-Based Systems

SCENARIO

A hospital's backward-chaining AI has 200 rules. It breaks every time a new virus appears and took 3 years to build. Diagnose WHY this keeps happening.

Task 1

Write 3 specific patient scenarios where backward chaining FAILS.
For each, name which of the 5 problems caused the failure.

Task 2

MATH: The hospital has 50 diseases and 30 symptoms.
How many rules are needed at minimum?
Show your calculation.

Task 3

In 2–3 sentences, propose what structure would fix this.
(Use intuition — no technical terms needed yet!)



SOLUTION 1.1: Diagnosing the Limits of Rule-Based Systems

Scenario 1 — New Virus

Patient has a new virus not in the rule database.
BC has no matching rule → chain never fires → no diagnosis.
CAUSE: Problem 1 (Bottleneck) + Problem 2 (no structural knowledge — can't infer 'similar to known virus').

Scenario 2 — Missing Symptom

Patient forgets to mention fatigue.
BC rule requires fever AND cough AND fatigue → fails silently.
CAUSE: Problem 3 (Brittleness with missing data).

Scenario 3 — Vocabulary Mismatch

Hospital B calls it 'Heart Attack', ours calls it 'MI'.
System cannot match despite being the same condition.
CAUSE: Problem 4 (No reuse / no shared vocabulary).

MATH ANSWER

50 diseases × 30 symptoms = 1,500 minimum rules
If each disease needs rules for symptom combinations: $2^{30} = \sim 1$ billion theoretically.
This is Problem 5: Combinatorial Explosion.



INSTRUCTOR REVEAL

Student intuition typically produces: 'A MAP that groups diseases by family'

That IS exactly a semantic network — what we study next!
You just reinvented the concept from first principles.



CLASS ACTIVITY 1.1-B: Real-World Case: Netflix Recommendation Engine Failure

REAL SCENARIO: Netflix's early system used IF-THEN rules: 'IF user watched Action THEN recommend Action'. It failed spectacularly. A user who watched Schindler's List (Drama/War) got recommended Die Hard (Action). Why?

Task 1

Draw a rule-based system with 5 IF-THEN rules for movie recommendation.

Then trace why 'IF watched-war-documentary THEN recommend comedy' could fire incorrectly.

Task 2

List 3 TYPES of relationships a movie has that pure rules miss:

- e.g. mood, time-period, director-style

Draw each as a node-edge pair.

Task 3

REDESIGN: Propose a semantic graph for ONE movie (your choice).

Include: genre, mood, era, actors, similar-to, director — connect with labeled edges.

Task 4

REAL-WORLD LINK:

How does Spotify's Discover Weekly solve this same problem with graphs?

Name 2 edge types Spotify likely uses in their knowledge graph.

✓ SOLUTION 1.1-B: Movie Knowledge Graph vs Rule-Based System

✗ Rule-Based System — Fails on Nuance

IF watched[Action] → recommend[Action]

IF watched[Drama] → recommend[Drama]

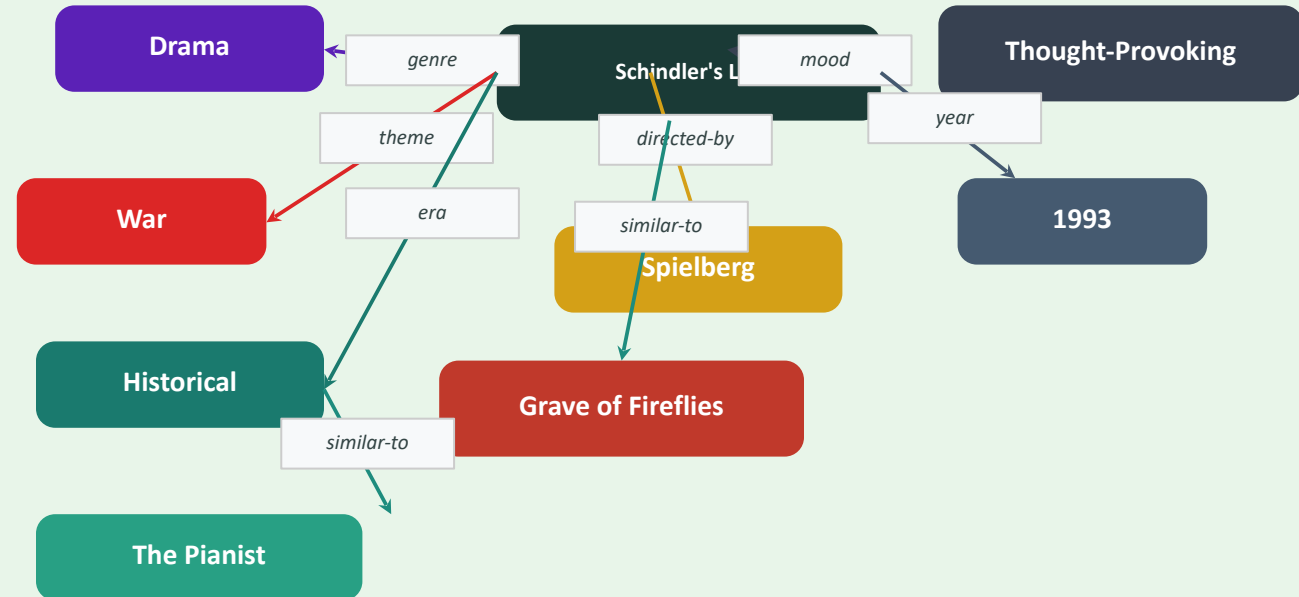
IF watched[Comedy] → recommend[Comedy]

FAILS: Schindler's List is War+Drama

→ gets Rule 2 → recommends Titanic

BUT user wanted Historical depth, NOT romance!

✓ Semantic Graph — Captures Real Relationships



INSIGHT: Spotify uses 'co-listened-with', 'same-era', 'same-mood' edges. Netflix uses 'similar-emotional-tone' and 'director-style' edges. The GRAPH captures human intuition that rules never could.

Semantic Networks and Graph-Based Knowledge

Directed graphs where structure IS the knowledge

Part 2 of 5

2.1 What Is a Semantic Network?

A semantic network is a **DIRECTED GRAPH** where **NODES** represent concepts and **EDGES** represent named relationships.

The key insight: the **STRUCTURE** of the graph **IS** the knowledge. You answer questions by traversing edges — no rules required.

NODES (Ovals)

- ▶ Root concept (e.g., Animal)
- ▶ Class level (e.g., Mammal)
- ▶ Sub-class (e.g., Dog)
- ▶ Instance — a real individual [Fido]
- ▶ Property / Action (Breathe, Fly)

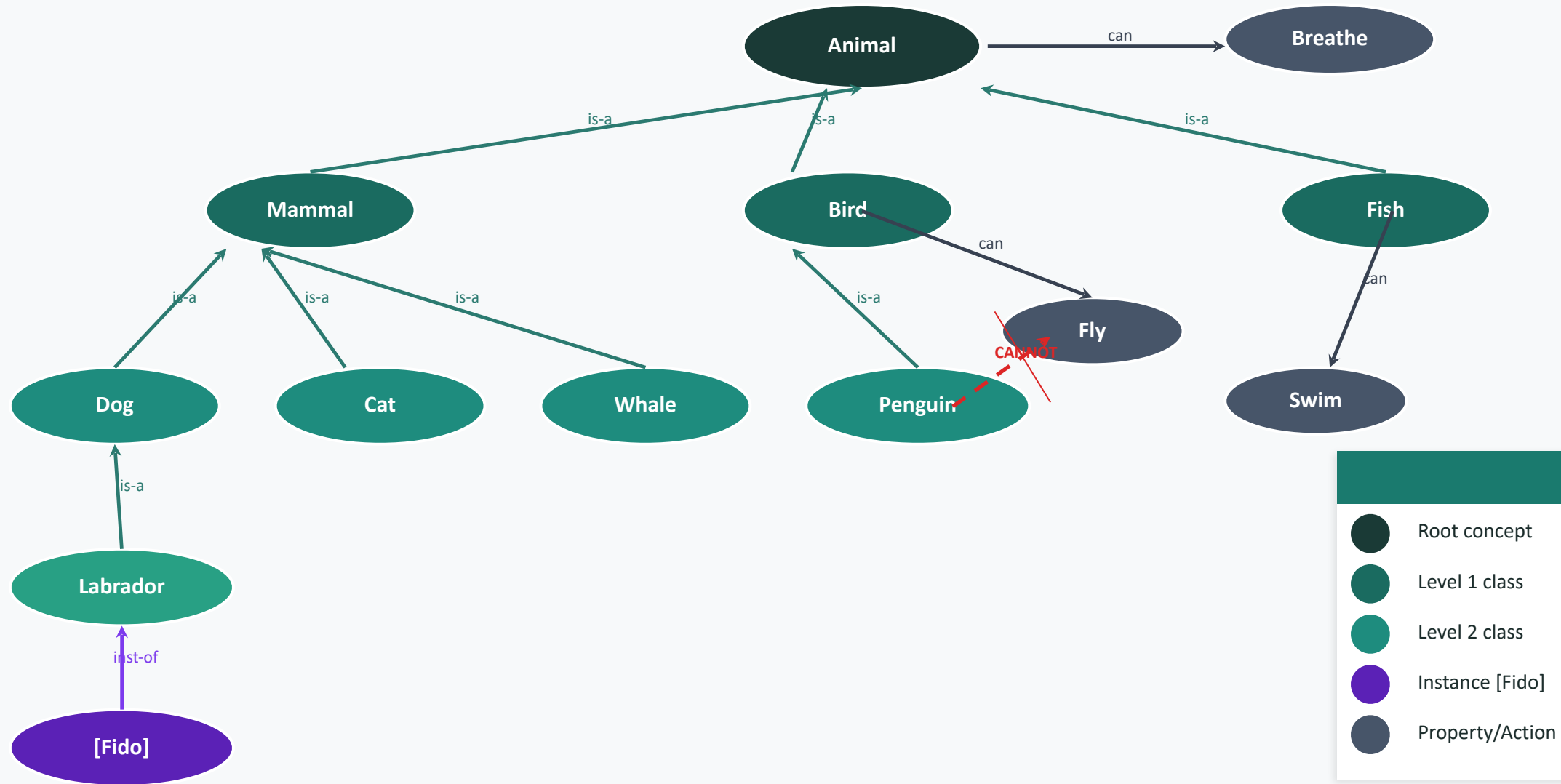
EDGES (Arrows)

- ▶ is-a → subclass relationship
- ▶ instance-of → specific individual
- ▶ has-a → part/component
- ▶ can → capability
- ▶ CANNOT → exception override

KEY PROPERTIES

- ▶ Inheritance: properties flow upward
- ▶ Transitivity: $A \rightarrow B, B \rightarrow C$ implies $A \rightarrow C$
- ▶ Specificity: direct edge beats inherited
- ▶ No rules needed — just traversal
- ▶ Scales to billions of facts (Google KG)

2.2 The Animal Kingdom Semantic Network



2.3 Graph Traversal — How Queries Are Answered

Query 1: Can Fido breathe?

TRAVERSAL PATH:

1. [Fido] → instance-of → Labrador
2. Labrador → is-a → Dog
3. Dog → is-a → Mammal
4. Mammal → is-a → Animal
5. Animal → can → Breathe ✓

ANSWER: YES — 4 hops, 0 rules

Query 2: Can Penguin fly?

TRAVERSAL PATH:

1. Penguin node — check direct edges first
2. FOUND: Penguin → CANNOT → Fly
3. STOP — direct edge found!
4. Specificity rule: Penguin's own edge beats inherited Bird edge

ANSWER: NO — 1 hop, specificity wins

Query 3: Is Fido a Mammal?

TRAVERSAL PATH:

1. [Fido] → instance-of → Labrador
2. Labrador → is-a → Dog
3. Dog → is-a → Mammal ✓
4. is-a is TRANSITIVE — chain works!

ANSWER: YES — 3 hops

2.4 Relationship Types Reference

Relationship	Meaning	Example
is-a	Subclass / type of	Dog is-a Mammal
instance-of	Specific individual (real-world)	[Fido] instance-of Dog
has-a	Part / component ownership	Dog has-a Tail
can	Capability / ability	Bird can Fly
CANNOT	Exception override (overrides inherited)	Penguin CANNOT Fly
lives-in	Location / habitat	Whale lives-in Ocean
made-by	Manufacturer / creator	iPhone made-by Apple
compatible-with	Works with / interoperability	iPhone compatible-with AirPods

2.5 Industry: Semantic Networks at Scale

Google Knowledge Graph (2012)



The Knowledge Panel on the right of search results IS a semantic network rendered live. When you search 'Albert Einstein', Google traverses edges to show: born in, spouse, awards, related topics — all from graph traversal.

LinkedIn Economic Graph



Maps 41K skills. Job recommendations come from graph traversal — not rules. 'You might know X' uses is-a and has-skill edges. 'Jobs that match you' traverses required-skill and has-skill edges.

Amazon Product Graph



Tax calculation, search filters, seller requirements, and 'Frequently Bought Together' all rely on semantic network traversal. A rule-based approach would require billions of explicit rules.



CLASS ACTIVITY 2.1: Build the E-Commerce Knowledge Graph

SCENARIO: Design a semantic network for an AI shopping assistant (like Amazon Alexa Shopping).

Task 1

Draw a semantic network with:

- 4+ levels of hierarchy
(Product → Electronics → Phone → iPhone)
- 8+ nodes total
- At least 5 different relationship types
- Label EVERY edge

Task 2

Trace paths for:

- a) 'Does iPhone have a battery?'
- b) 'Is a Laptop a product?'
- c) 'What can a Smartwatch do?'

Write out every node and edge in the path.

Task 3

Add an EXCEPTION:

Earbuds are Electronics BUT cannot-make-calls.

Draw the override edge and explain how specificity handles it.

Task 4

CHALLENGE: 'What products are compatible with iPhone 15?'

Trace the path. Which type of relationship do you follow?



SOLUTION 2.1: E-Commerce Graph — Path Answers

Path (a): Does iPhone have a battery?

[iPhone15] → instance-of → iPhone
→ is-a → Phone → is-a →
Electronics → has-a → Battery

ANSWER: YES (4 hops, 0 rules
needed!)

Path (b): Is a Laptop a product?

Laptop → is-a → Electronics → is-a
→ Product

ANSWER: YES (2 hops — short
chain)

Path (c): What can a Smartwatch do?

Start at Smartwatch, follow ALL
outgoing 'can' edges:
can → Track-Health
can → Notify
can → Tell-Time

Return all 3 capabilities.

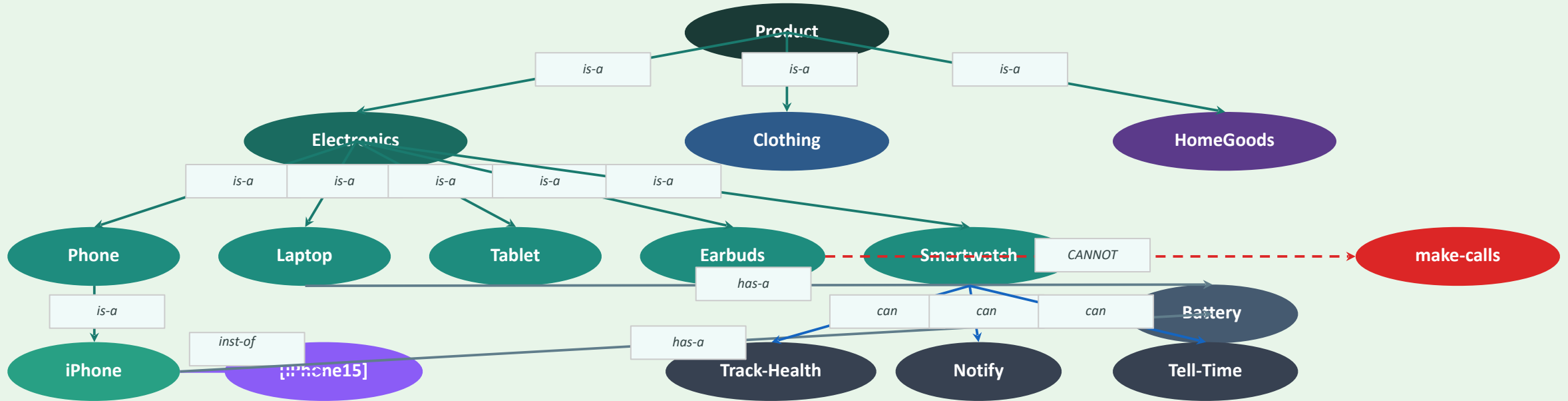
Exception: Earbuds CANNOT make-calls

Add direct edge: Earbuds →
CANNOT → make-calls

This OVERRIDES any inherited call
capability.
Specificity wins — direct edge is
most specific.

✓ SOLUTION 2.1: E-Commerce Knowledge Graph — Full Diagram

● Category node ● Sub-type ● Instance —▶ is-a (teal) ..▶ has-a / can (gray/blue) - -▶ CANNOT exception (red)



✓ [iPhone15] -inst-of→ iPhone -is-a→ Phone -is-a→ Electronics -has-a→ Battery (4 hops, ZERO if-then rules)

KEY LEARNING: The graph answered ALL queries with zero IF-THEN rules. The edges ARE the knowledge. This is why graph-based KR scales to 350M+ products when rule-based AI cannot.



CLASS ACTIVITY 2.2: Semantic Network Reasoning Race

GIVEN NETWORK:

Animal→can→Breathe | Mammal→is-a→Animal | Dog→is-a→Mammal | Cat→is-a→Mammal | Bird→is-a→Animal | Bird→can→Fly | Penguin→is-a→Bird | Penguin→CANNOT→Fly | Whale→is-a→Mammal | Whale→lives-in→Ocean | Flipper→instance-of→Dolphin | Dolphin→is-a→Mammal

(a) Can Flipper breathe?

Trace path:

Edges traversed: ____

(b) Does Whale live in Ocean?

Trace path:

Edges traversed: ____

(c) Can Cat fly?

Trace path:

Edges traversed: ____

(d) Is Penguin an Animal?

Trace path:

Edges traversed: ____

(e) Can Penguin fly?

Trace path:

Edges traversed: ____

(f) What can Bird do?

Trace path:

Edges traversed: ____

(g) Is Flipper a Mammal?

Trace path:

Edges traversed: ____

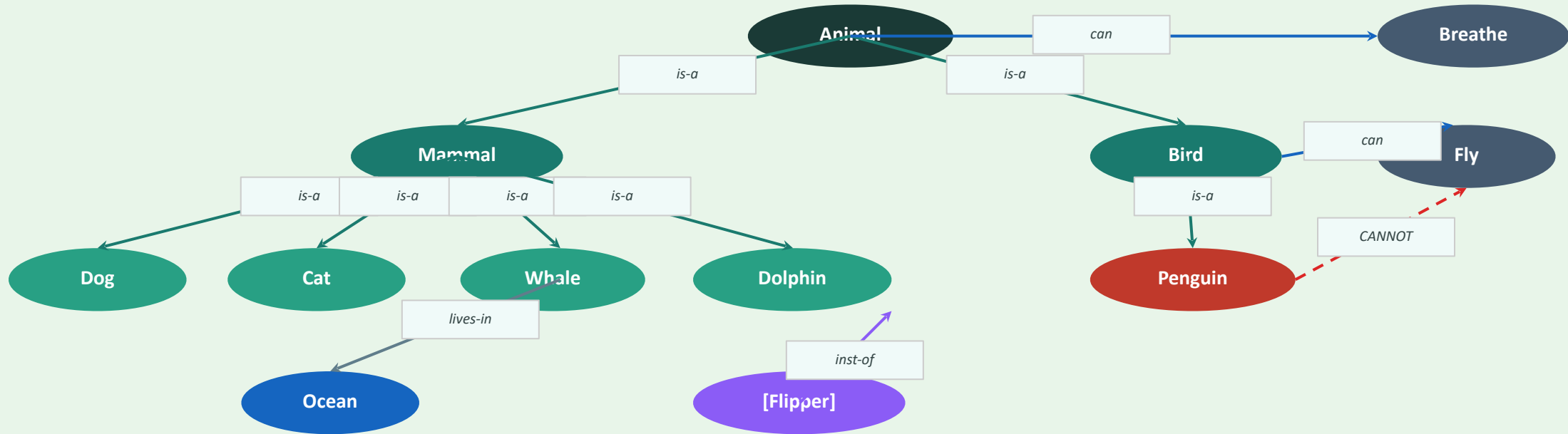
(h) Can Whale breathe?

Trace path:

Edges traversed: ____

✓ SOLUTION 2.2: Semantic Network Reasoning Race — Full Network Diagram

—▶ is-a (teal) —▶ can (blue) —▶ lives-in (gray) —▶ CANNOT (red) —▶ instance-of (purple)



YES

(a) Flipper breathe?

→Dolphin→Mammal→Animal→can→Breathe

YES

(b) Whale in Ocean?

Whale→lives-in→Ocean (direct)

NO

(c) Cat fly?

Cat→Mammal→Animal — no Fly edge

YES

(d) Penguin=Animal?

Penguin→Bird→Animal

✓ SOLUTION 2.2: Semantic Network Reasoning Race — All 8 Answers

(a) Can Flipper breathe?

YES

Flipper→Dolphin→Mammal→Animal→Breathe

4 hops

(b) Does Whale live in Ocean?

YES

Whale→lives-in→Ocean (direct edge)

1 hop

(c) Can Cat fly?

NO

Cat→Mammal→Animal — no Fly edge found. Bird is NOT an ancestor of Cat.

3 hops, no fly found

(d) Is Penguin an Animal?

YES

Penguin→Bird→Animal

2 hops

(e) Can Penguin fly?

NO

Penguin→CANNOT→Fly (direct edge — stops immediately)

1 hop — specificity!

(f) What can Bird do?

2
abilities

Fly (Bird→can→Fly) + Breathe (inherited from Animal)

2 capabilities

(g) Is Flipper a Mammal?

YES

Flipper→Dolphin→Mammal

2 hops

(h) Can Whale breathe?

YES

Whale→Mammal→Animal→can→Breathe

3 hops



CLASS ACTIVITY 2.3: Real-World Case: Google Knowledge Panel Design

REAL SCENARIO: When you Google 'Albert Einstein', Google shows a Knowledge Panel on the right — birthdate, nationality, spouse, field, awards, related people. This is a LIVE semantic network query. Today you design it.

Task 1

Design the semantic network for a PERSON node.

Required nodes: Albert Einstein, Physics, Nobel Prize, Germany, $E=mc^2$, Relativity.

Label every edge with the relationship type.

Task 2

TRAVERSAL QUERY:
User types: 'Who else won Physics Nobel?'

Trace the EXACT graph path starting from Einstein node.
How many hops? Which edge types?

Task 3

ADD COMPLEXITY:
Einstein is 'related-to' Niels Bohr, Marie Curie.

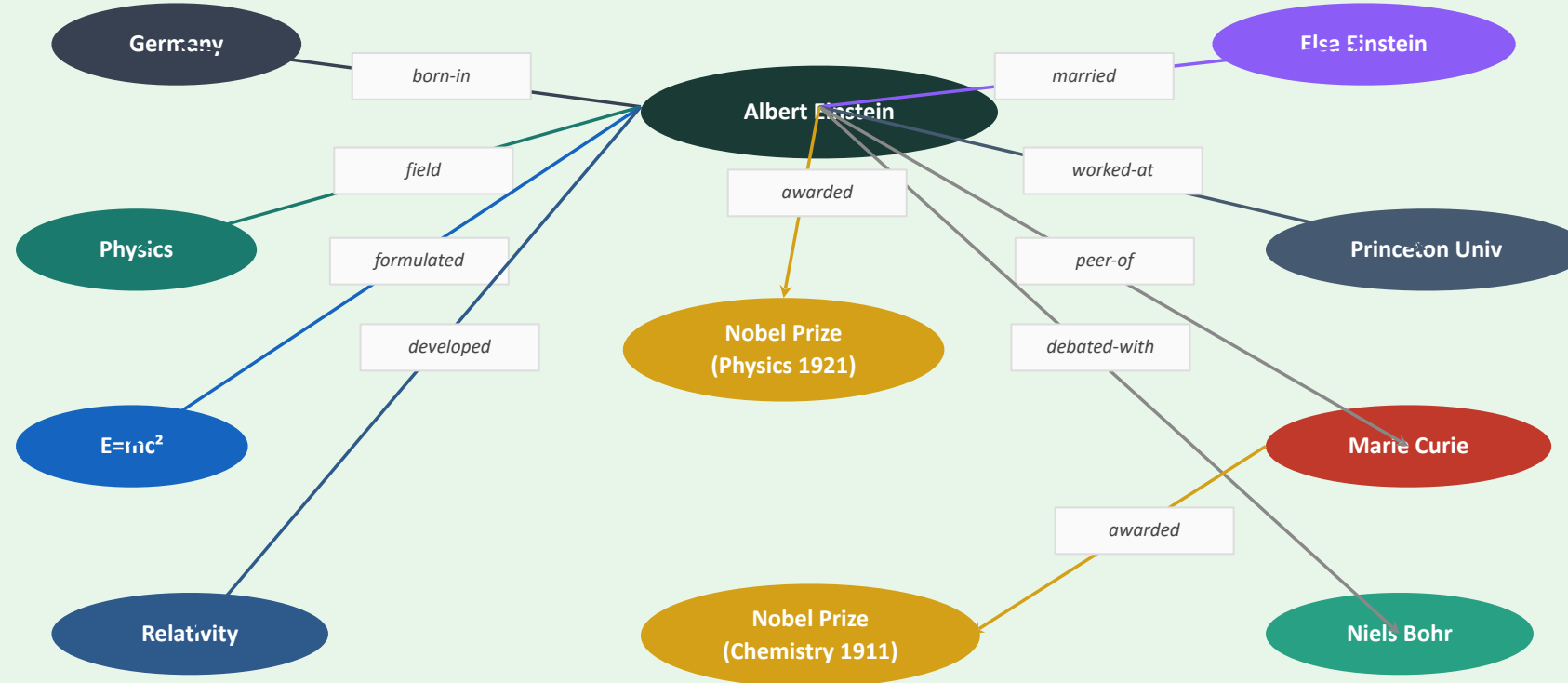
Both also won Nobel Prizes. Draw the subgraph connecting all three and the Nobel Prize node.

Task 4

CRITICAL THINKING:
Google's Knowledge Graph has 500 BILLION facts.

Why can Google query 'Who was Einstein's teacher?' in 0.2 seconds? Would IF-THEN rules work at this scale?

✓ SOLUTION 2.3: Google Knowledge Graph — Einstein Subgraph



🔍 QUERY: 'Who else won Physics Nobel?' → Einstein —awarded→ Nobel Prize(Physics1921) ←awarded— [traverse all people with same edge to same node] → Bohr (1922), Dirac (1933)...

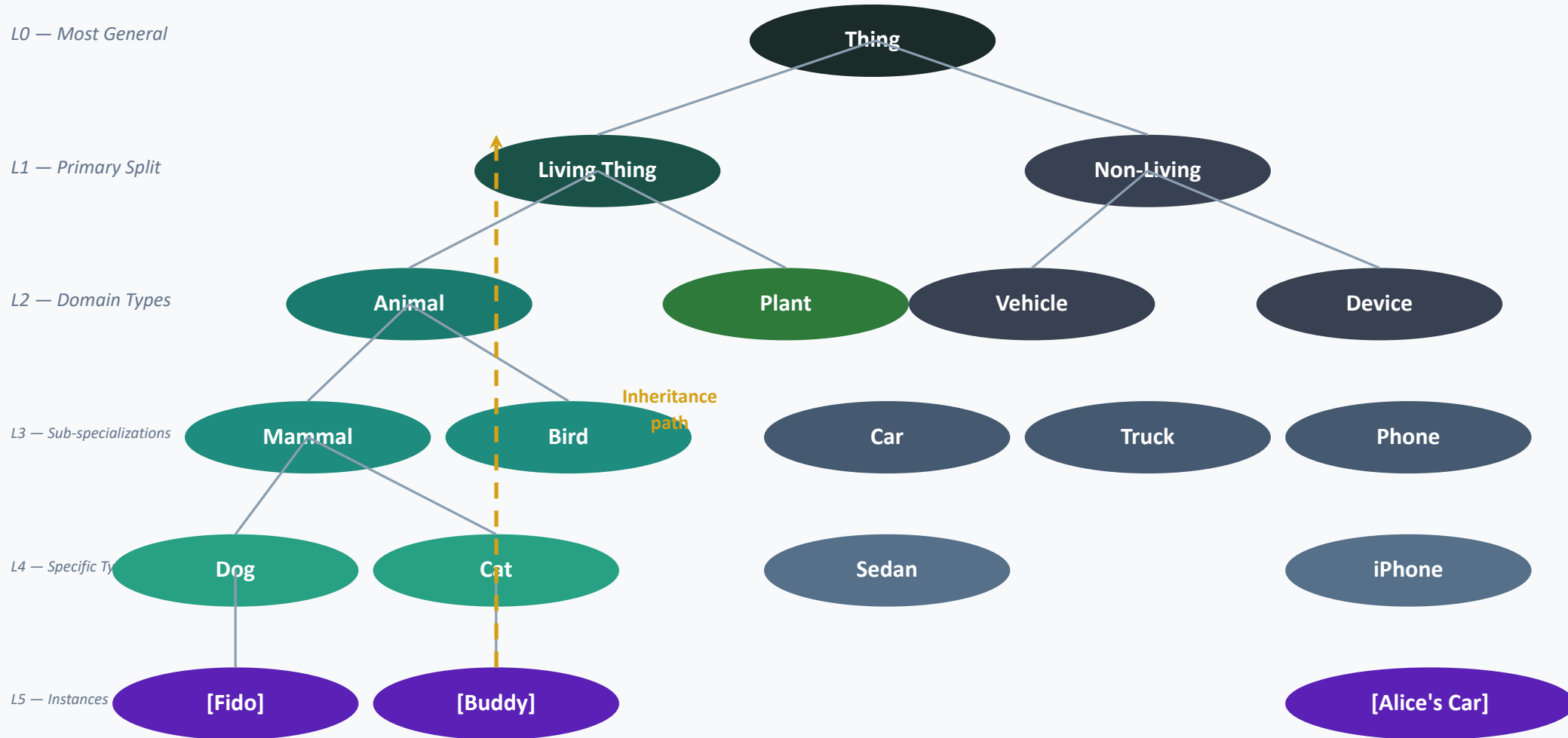
SCALE INSIGHT: Google's 500B-fact graph answers this in 0.2s because it's ONE edge traversal. 500B IF-THEN rules checking every possibility would take hours. Graph traversal = $O(\text{edges})$, rule checking = $O(\text{rules} \times \text{facts})$.

Concept Hierarchies and Inheritance

From general to specific — properties flow downward

Part 3 of 5

3.1 What Is a Concept Hierarchy?



3.2 Inheritance — Properties Flow Downward



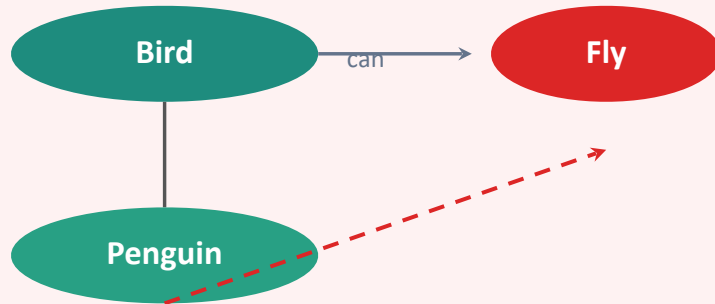
What [Buddy] Gets for FREE

- **has-cells = TRUE**
← from Animal (4 levels up)
- **warm-blooded = TRUE**
← from Mammal (3 levels up)
- **barks = TRUE**
← from Dog (2 levels up)
- **guide-capable = TRUE**
← from Labrador (1 level up)
- **name = Buddy**
← Buddy's OWN property

WITHOUT inheritance: 5 facts stored explicitly.
WITH inheritance: 1 fact stored (name = Buddy). 4 come free!

3.3 Exception Handling — The Specificity Principle

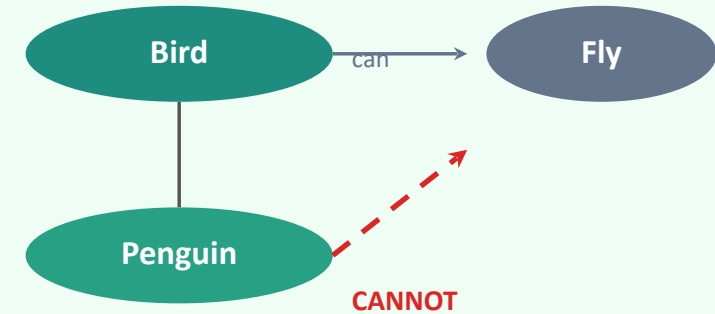
✗ WITHOUT Exception Handling



WRONGLY inherits
can-fly = TRUE!

Penguins are birds → Penguins CAN fly (WRONG!)
Rule-based and naive graph systems make this mistake.

✓ WITH the Specificity Principle



SPECIFICITY RULE:

The MOST SPECIFIC edge (closest to the queried node) ALWAYS wins.
Penguin's direct CANNOT edge overrides Bird's inherited CAN edge.

WHY THIS MATTERS: Store the GENERAL RULE once (Birds can fly). Add INDIVIDUAL EXCEPTIONS where needed (Penguin). You do NOT redefine everything from scratch for each exception!



CLASS ACTIVITY 3.1: Design a University Course Hierarchy

SCENARIO: An AI advisor needs to answer: 'Is this course technical?', 'Does it have an assessment?', 'Are prerequisites required?' Design a 5-level concept hierarchy for university courses.

Task 1

Draw a 5-level node-edge tree:

- Academic Course at top
- 3 category branches
- 6 specific leaf courses
- Connect with is-a edges

Task 2

Define 3 inherited properties at each top level.

Trace which properties flow down to leaf courses by following inheritance paths.

Task 3

Add 2 exceptions:

- One course that OVERRIDES an inherited property

Example: A thesis course overrides has-exam = FALSE

Task 4

Trace these queries:

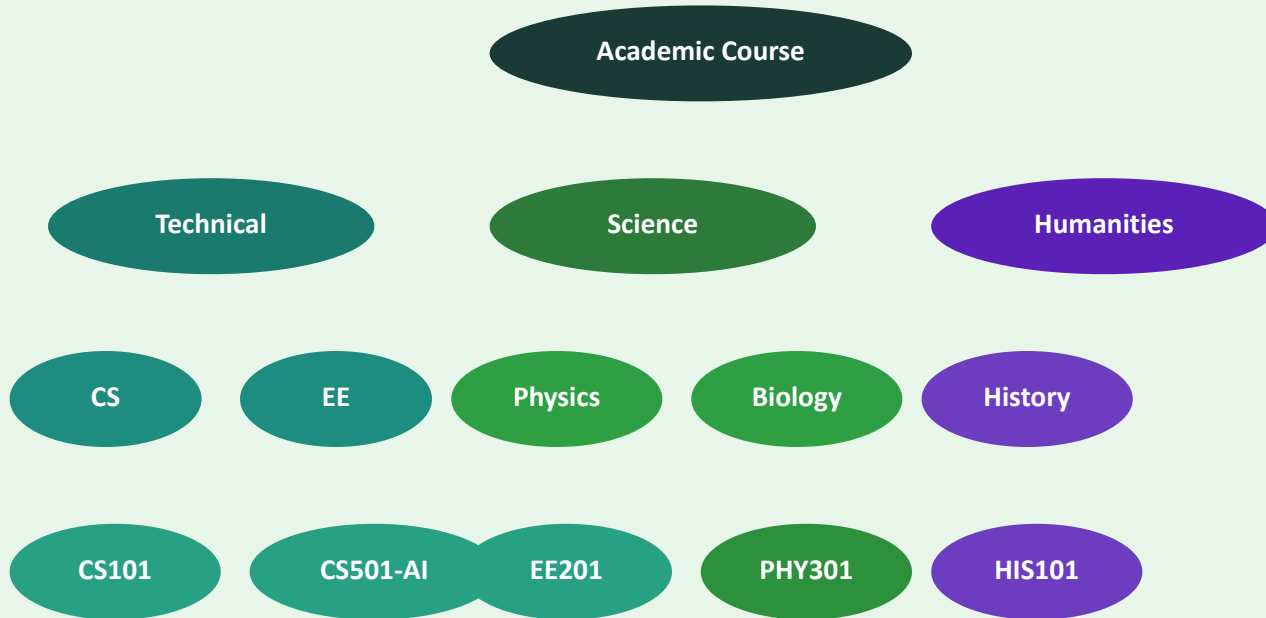
- 'Is CS101 technical?'
- 'Does CS101 have an assessment?'

Show exact node-edge path for each.



SOLUTION 3.1: Design a University Course Hierarchy

HIERARCHY:



Properties: Academic Course: has-assessment=TRUE, has-credits=TRUE, has-syllabus=TRUE
Technical: involves-coding=TRUE, has-lab=TRUE | Science: involves-experiments=TRUE
EXCEPTION: CS999-Thesis overrides has-exam=FALSE (direct edge beats inherited TRUE)

TRACE: 'Is CS101 technical?'

CS101 → is-a → Computer Science → is-a → Technical

ANSWER: YES. 2 hops.

TRACE: 'Does CS101 have assessment?'

CS101 → no own has-assessment edge

↑ go up → Computer Science → no edge

↑ go up → Technical → no edge

↑ go up → Academic Course → has-assessment = TRUE ✓

ANSWER: YES — inherited from 3 levels up!



CLASS ACTIVITY 3.2: Inheritance Debugging Challenge

SCENARIO: Find what is REDUNDANT, WRONG, or MISSING in this knowledge base. Classify each fact, write the minimal correct set.

(a) Animal.can-breathe = TRUE

Classification: _____

(b) Dog.can-breathe = TRUE

Classification: _____

(c) Labrador.can-breathe = TRUE

Classification: _____

(d) Cat.can-breathe = TRUE

Classification: _____

(e) Bird.can-fly = TRUE

Classification: _____

(f) Penguin.can-fly = TRUE

Classification: _____

(g) Penguin.is-a = Bird

Classification: _____

(h) Bat.is-a = Mammal

Classification: _____

(i) Bat.can-fly = NOT DEFINED

Classification: _____



SOLUTION 3.2: Inheritance Debugging Challenge

CORRECT & NEEDED

(a) `Animal.can-breathe=TRUE`

Source of breathing — remove it and the property disappears entirely.

REDUNDANT

(b) `Dog.can-breathe=TRUE`

Dog → Mammal → Animal already inherits it. Delete!

REDUNDANT

(c) `Labrador.can-breathe=TRUE`

Inherits through 3 levels. Delete!

REDUNDANT

(d) `Cat.can-breathe=TRUE`

Cat → Mammal → Animal covers it. Delete!

CORRECT & NEEDED

(e) `Bird.can-fly=TRUE`

Default for all birds. Must stay.

✗ WRONG!

(f) `Penguin.can-fly=TRUE`

Penguins cannot fly! Should be: `Penguin.can-fly=FALSE` (explicit override).

CORRECT & NEEDED

(g) `Penguin.is-a=Bird`

Structural relationship. Must stay.

CORRECT & NEEDED

(h) `Bat.is-a=Mammal`

Bats are mammals. Must stay.

✗ MISSING!

(i) `Bat.can-fly=UNDEFINED`

Should be: `Bat.can-fly=TRUE`. Bats fly despite being mammals.

MINIMAL CORRECT SET: (a), (e), FIXED-(f) as FALSE, (g), (h), NEW-(i) as TRUE = 6 facts. REDUCTION: 9 → 6 facts = 33% saved!

Frames and Slot-Based Knowledge Structures

Minsky's templates: structured records with active procedures

Part 4 of 5

4.1 What Is a Frame?

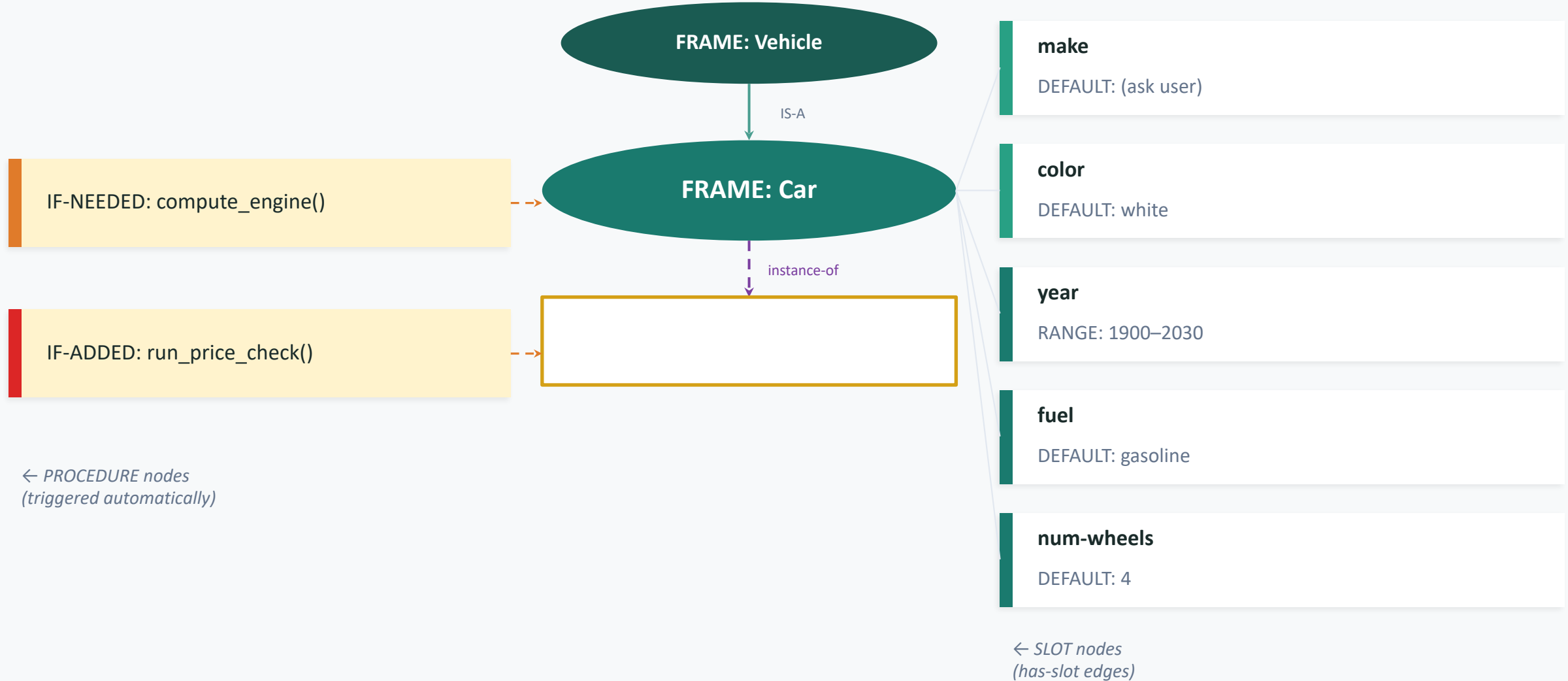
Minsky (1975): Human memory uses **TEMPLATES** — mental structures pre-loaded with typical knowledge.

When you walk into a restaurant you've never been to, you already know: there will be a menu, someone will take your order, you will pay at the end. This template is a **FRAME**.

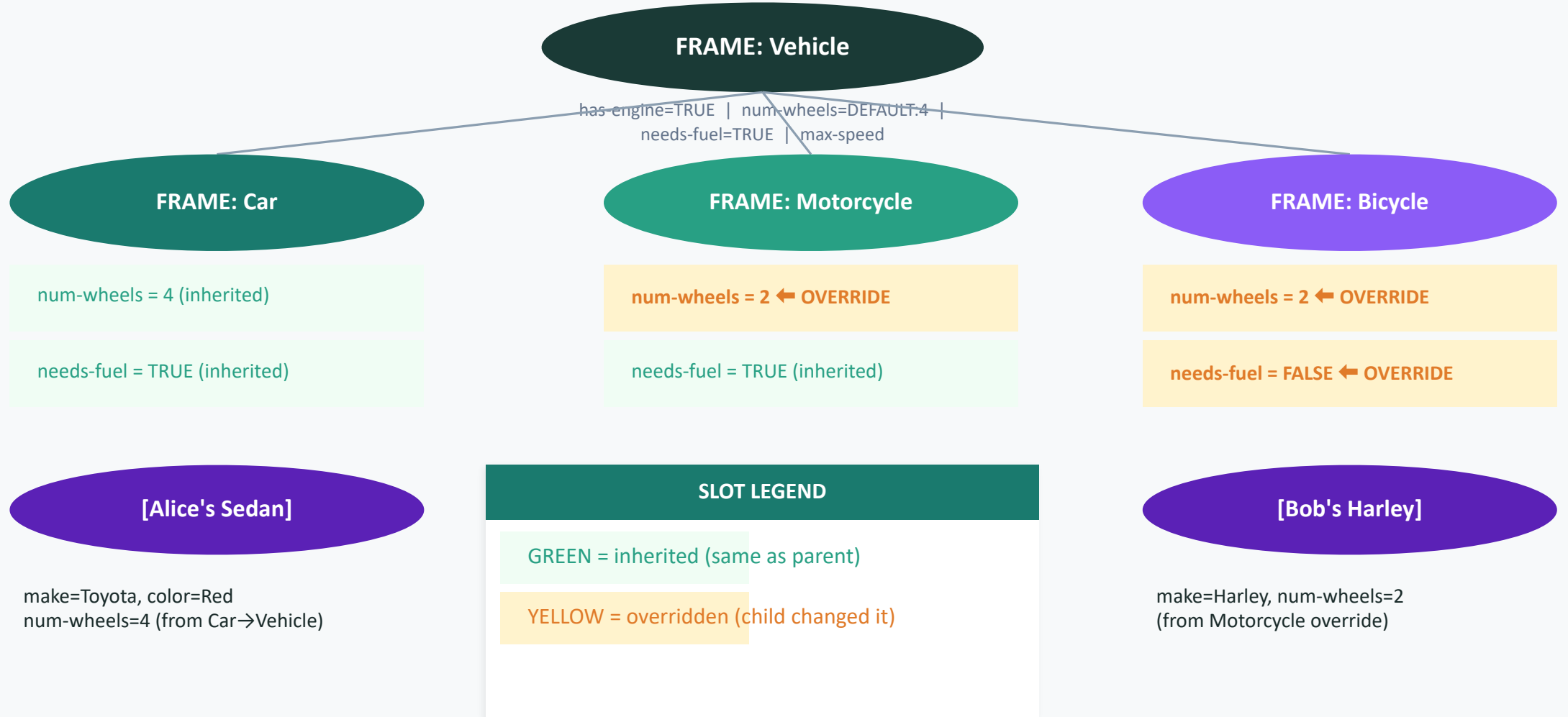
SLOT NAME	VALUE	DEFAULT	RANGE CONSTRAINT
Named attribute of the concept (make, color, year, fuel)	The actual stored value (make = Toyota)	Value if none given (fuel = gasoline)	Valid values / data type (year: 1900–2030)
IF-NEEDED	IF-ADDED	IF-REMOVED	
Computed on demand compute_engine(year, model)	Fires when value is set run_price_check()	Fires when value deleted archive_record()	

4.2 The Car Frame — Node Diagram

Hour 4



4.3 Frame Inheritance — The Vehicle Family



4.4 Procedural Attachments — 3 Types

IF-NEEDED

Computes a slot's value ON DEMAND.
Never stored — re-computed each time.
Like a spreadsheet formula.

```
Employee.age:  
IF-NEEDED → compute_from(birth_date,  
today)
```

Python Analogy:

```
@property  
def age(self):  
    return today - self.birth_date
```

IF-ADDED

Runs when a slot gets a NEW value.
Like a database trigger / setter with side effects.
Automatically sets up related data.

```
Employee.department:  
IF-ADDED → create_Slack_channel() +  
assign_mentor()
```

Python Analogy:

```
@department.setter  
def department(self, dept):  
    create_slack(dept)  
    assign_mentor(self.role)
```

IF-REMOVED

Runs when a slot value is DELETED.
Cleans up related resources automatically.
Ensures consistency across systems.

```
Employee.end_date:  
IF-REMOVED → deactivate_access_card()  
+ archive_email()
```

Python Analogy:

```
def remove_employee(emp):  
    deactivate_badge(emp.id)  
    archive_email(emp.id)
```



CLASS ACTIVITY 4.1: Employee Onboarding Frame System

SCENARIO: Design an AI-powered HR system for a 500-person tech firm. When a new hire joins, auto-setup accounts, compute tax, assign a mentor, schedule orientation.

Task 1

Design FRAME: Employee with 10+ slots.

For each slot, define:

- Name
- Type / range
- Default value (if any)
- Any procedure (IF-NEEDED, IF-ADDED)

Task 2

Create 2 sub-frames:

- FullTimeEmployee (IS-A Employee)
- Contractor (IS-A Employee)

Which slots differ? What do they override?

Task 3

Create INSTANCE [NewHireAhmed]:

- Fill all slots you know.
- For empty slots, which default fires?
- Which procedures run automatically?

Task 4

Write 3 procedures:

- a) IF-ADDED for 'department' slot
- b) IF-NEEDED for 'years-experience'
- c) IF-REMOVED for 'end-date'



SOLUTION 4.1: Employee Onboarding Frame System

FRAME: Employee — Key Slots

Slot	Default / Procedure
employee-id	IF-ADDED: auto-assign ID
email	IF-NEEDED: name@company.com
department	IF-ADDED: create Slack + assign mentor
salary	IF-ADDED: compute-tax()
end-date	DEFAULT: null; IF-REMOVED: deactivate-access
years-experience	IF-NEEDED: today – start_date
role	Required — no default

Sub-frames: FullTimeEmployee vs Contractor

FullTimeEmployee ADDS: benefits=standard, annual-leave=21, pension (IF-ADDED: enroll-pension)

Contractor OVERRIDES: benefits=NONE, annual-leave=0
Contractor ADDS: contract-end-date (required), hourly-rate

Procedure Pseudocode

```
IF-ADDED department:  
    create_slack(dept), assign_mentor(role, dept),  
    add_to_org_chart()
```

```
IF-NEEDED years_experience:  
    return (today – start_date).years
```

```
IF-REMOVED end_date:  
    deactivate_badge(id), archive_email(id),  
    generate_exit_report(id)
```



CLASS ACTIVITY 4.2: Hospital Triage Frame Reasoning

SCENARIO: Use frame reasoning (slots, defaults, procedures) to triage 3 patients below.

PATIENT A

age = 67
symptoms = [chest-pain,
shortness-of-breath]
BP = UNFILLED
allergies = UNFILLED

PATIENT B

age = 25
symptoms = [headache, fever=38.5°C]
BP = 120/80
allergies = [penicillin]

PATIENT C

age = 8
symptoms = [broken-arm]
BP = UNFILLED
pain-scale = 7

Task 1

For each patient:
1. Apply defaults to UNFILLED slots
2. Identify which IF-NEEDED fires
3. Determine triage priority (CRITICAL / HIGH / MODERATE)

Task 2

Write IF-NEEDED pseudocode for triage-priority.

What inputs does it need?
What score thresholds map to each priority?

Task 3

DISCUSS: MISSING data vs WRONG data.
How does each affect the frame system?
Which is more dangerous in medicine?



SOLUTION 4.2: Hospital Triage Frame Reasoning

PATIENT A → CRITICAL

BP=UNFILLED → default: 'needs-measurement'
allergies=UNFILLED → default: 'none-known'
IF-NEEDED fires: age=67 (HIGH) + chest-pain +
shortness-of-breath (cardiac flags)
PRIORITY: CRITICAL

PATIENT B → MODERATE

All slots filled.
IF-NEEDED: age=25 (low risk) + fever=38.5
(moderate) + no cardiac symptoms
Store allergy=penicillin for drug-prescription check
later.
PRIORITY: MODERATE

PATIENT C → HIGH

BP=UNFILLED → default: 'needs-measurement'
IF-NEEDED: age=8 (child, extra care) + broken-arm
(injury) + pain=7 (significant)
PRIORITY: HIGH

IF-NEEDED Pseudocode: triage-priority

```
score = 0
IF age > 60:  score += 30
IF 'chest-pain' in symptoms:  score += 50
IF 'shortness-of-breath' in symptoms:  score += 40
IF fever > 39:  score += 20
IF age < 12:  score += 15
IF score > 60: CRITICAL | > 30: HIGH | ELSE: MODERATE
```

MISSING vs WRONG Data

MISSING data: default kicks in gracefully. System CONTINUES working with conservative safe assumption.

WRONG data: system uses the wrong value with full confidence. No error detected. This is DANGEROUS in medicine!

This motivates ONTOLOGY CONSTRAINTS — our next topic.



CLASS ACTIVITY 5.2: Real-World Case: Wikidata Ontology — RDF Triples in Practice

REAL SCENARIO: Wikidata (used by Wikipedia, Google, Siri) stores ALL world knowledge as RDF triples. Every fact is: Subject → Predicate → Object. Right now, over 100 million such triples describe people, places, species, and events. Today you write and query them.

Task 1

Write 10 RDF triples for a Pakistani cricket player (e.g. Babar Azam).

Format: <Subject> <Predicate> <Object>

Include: nationality, birthdate, role, team, achievements, type of person.

Task 2

OWL CONSTRAINT:

Write constraints that ensure:

- A player MUST have a team
- A team MUST have a country
- A match-score must be an Integer
- dateOfBirth must be before today

Use OWL-style pseudocode.

Task 3

SPARQL-STYLE QUERY:

Write the English logic for:

'Find all players who play for Pakistan AND have scored a century'

Then show which triples you'd traverse and in what order.

Task 4

OPEN WORLD ASSUMPTION:

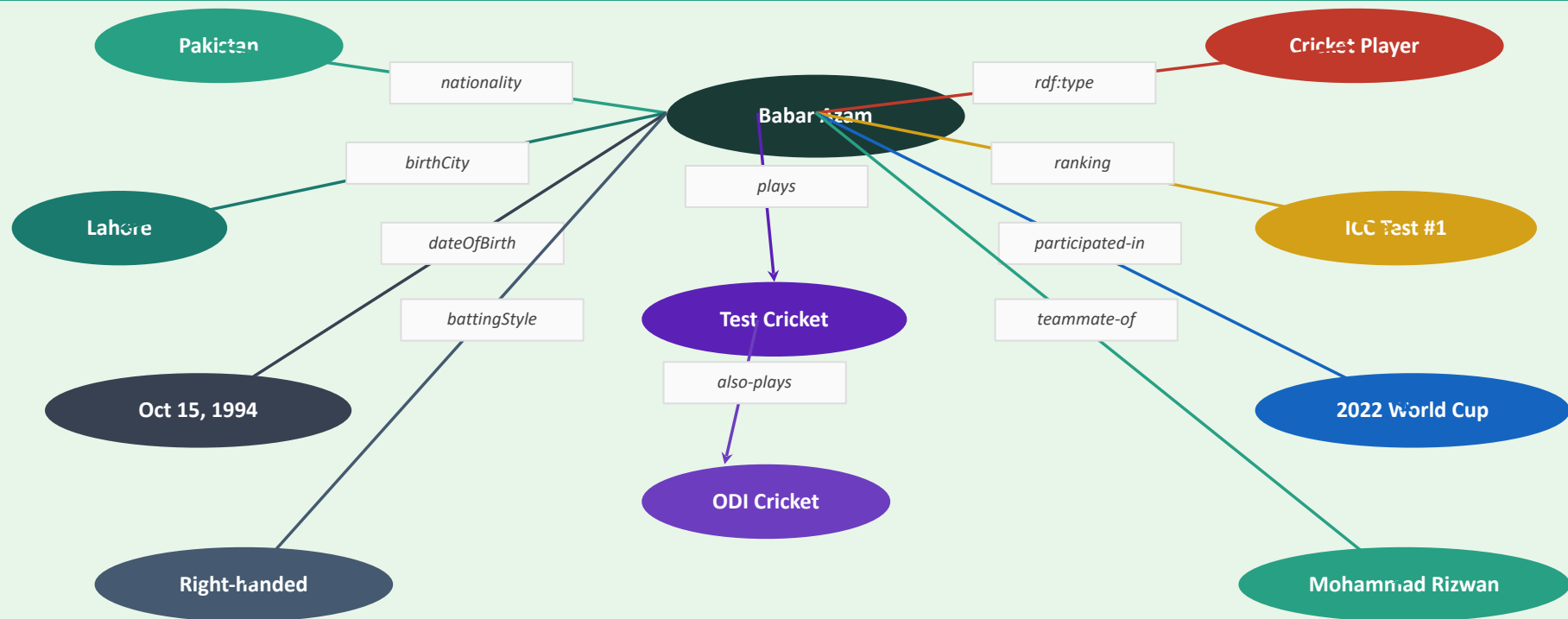
Wikidata does NOT know Babar's shoe size.

In Closed World: 'shoe-size = UNKNOWN means FALSE'

In Open World: 'shoe-size = not yet stated'

Why does this matter for AI reasoning? Give 2 examples where the wrong assumption causes errors.

✓ SOLUTION 5.2: Wikidata RDF Graph — Babar Azam Knowledge Network



Sample RDF Triples (Subject → Predicate → Object)

<BabarAzam> <nationality> <Pakistan>		<BabarAzam> <rdf:type> <CricketPlayer>
<BabarAzam> <birthCity> <Lahore>		<BabarAzam> <ranking> <ICC-Test-#1>
<BabarAzam> <plays> <TestCricket>		<Pakistan> <rdf:type> <Country>

OPEN WORLD: Wikidata doesn't know Babar's shoe size → it's simply 'not stated', NOT 'FALSE'. This lets AI say 'I don't know' instead of wrong confident answers. Closed World would dangerously conclude shoe-size=0.



CLASS ACTIVITY 4.3: Real-World Case: Airbnb Property Knowledge Frame

REAL SCENARIO: Airbnb uses frames (they call them 'structured data schemas') for every property listing. When you search 'pet-friendly 2-bedroom near beach', Airbnb's AI traverses FRAMES of millions of properties. Today you design one.

Task 1

Design FRAME: AirbnbProperty
With these slot categories:

- Identity slots (name, host-id, location)
- Physical slots (bedrooms, max-guests, amenities)
- Policy slots (pet-friendly, smoking, check-in-time)
- Procedure slots (IF-NEEDED, IF-ADDED)

Task 2

Create sub-frames:

- Apartment IS-A AirbnbProperty
- Villa IS-A AirbnbProperty
- HotelRoom IS-A AirbnbProperty

Which slots differ? E.g. Villa adds: pool, garden. HotelRoom removes: kitchen.

Write the overrides.

Task 3

INSTANCE: [Lahore-Gulberg-2BR]

- Fill all known slots
- Which slots fire IF-NEEDED?
- Write the procedure for:
'IF-ADDED booking-date → check-availability, update-calendar, send-confirmation'

Task 4

SEARCH QUERY TRACE:

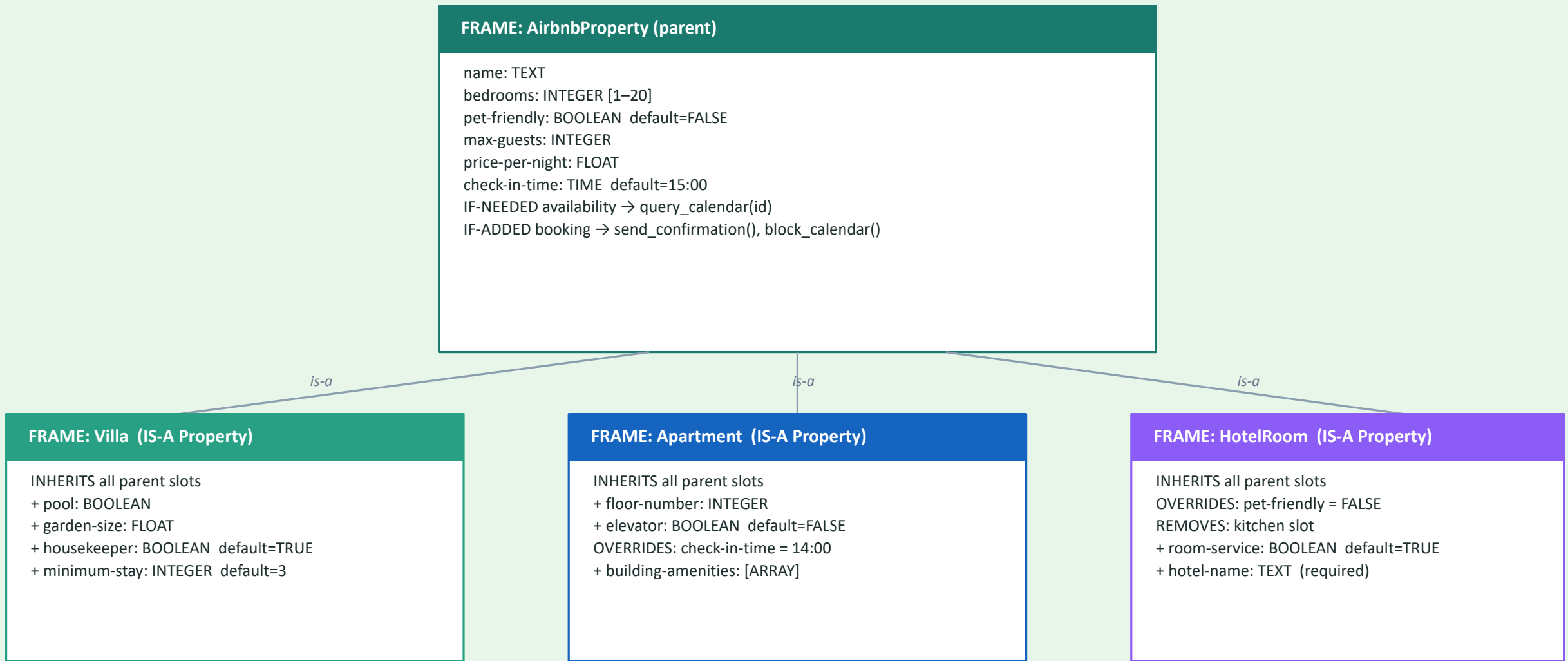
User searches: 'pet-friendly 2-bed near Lahore under \$80'

Trace how the frame system:

1. Filters by pet-friendly slot
2. Filters bedrooms ≥ 2
3. Calculates distance
4. Checks price slot

How many frame lookups per property?

✓ SOLUTION 4.3: Airbnb Property Frame — Inheritance Diagram



INSIGHT: Airbnb has 7M+ listings. Each is an INSTANCE of one of these frames. A search query checks 3–4 slot values per listing = ~28M slot lookups vs billions of IF-THEN rule evaluations. Frames win at scale.

Introduction to Ontologies

Formal, shared, machine-readable knowledge — OWL & RDF

Part 5 of 5

5.1 What Is an Ontology?

Hour 5

Ontology = VOCABULARY + STRUCTURE + RULES

written in a machine-readable formal language that multiple systems can share and reason with.

FORMAL

Written in a precise language (OWL/RDF) — not ambiguous natural language. Computers can parse and reason with it.

EXPLICIT

Every concept, relationship, and constraint is stated clearly — nothing is implied or assumed.

SHARED

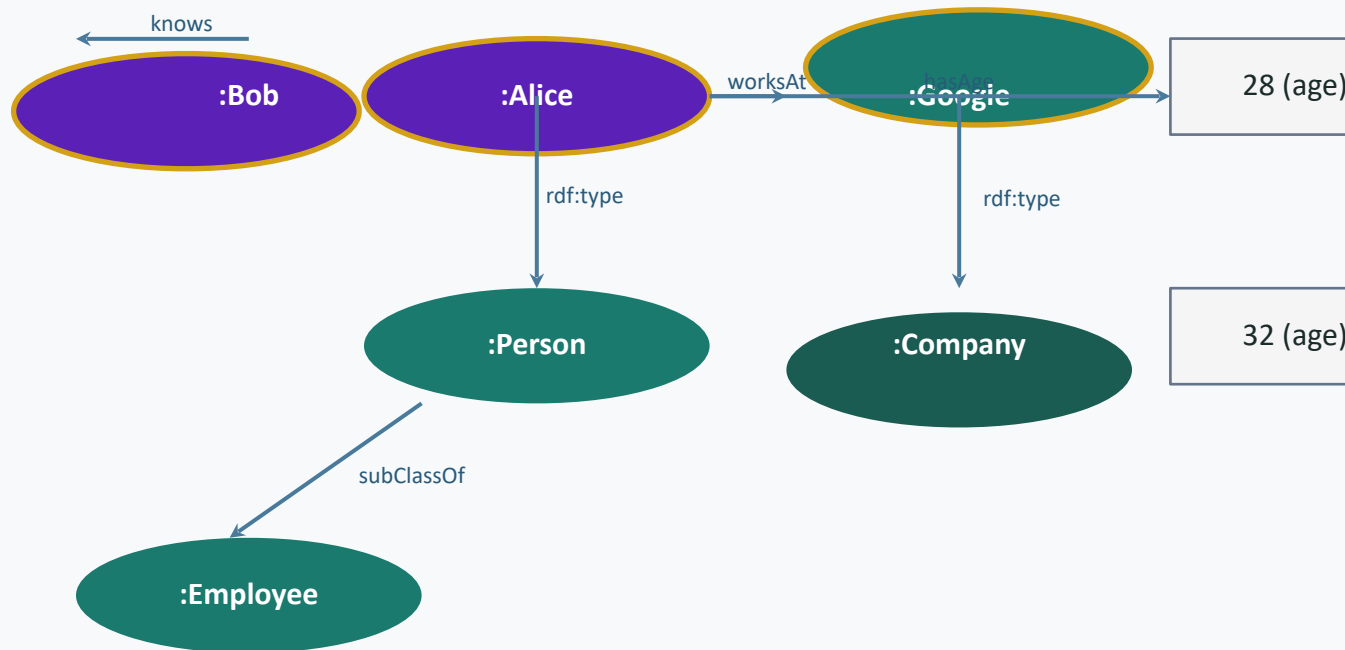
Agreed upon by multiple organisations — one single source of truth for a domain.

CONCEPTUALIZATION

Models a domain of reality — not reality itself. An abstraction over the world.

MEMORY HOOK: Ontology = Semantic Network + Frames + Formal Logic + Standards

It takes everything we've studied and adds: standardisation, formal constraints, automated reasoning, and cross-organisation interoperability.



Every Arrow = One RDF Triple

:Alice —**rdf:type**→ **:Person**

→ Alice IS a Person

:Alice —**worksAt**→ **:Google**

→ Alice WORKS AT Google

:Alice —**hasAge**→ **28**

→ Age = data property (literal)

:Employee —**subClassOf**→ **:Person**

→ All Employees are Persons

KEY INSIGHT: This graph IS a semantic network expressed in a formal standard. Anyone in the world using RDF can read, query (with SPARQL), and reason over it.

subClassOf

Every Employee IS a Person.
All instances of Employee are automatically instances of Person.

```
:Employee subClassOf :Person
```

disjointWith

Man $\perp$ Woman. Nothing can be BOTH.
If :Alice typed as both $\rightarrow$ CONTRADICTION auto-detected!

```
:Man disjointWith :Woman
```

minCardinality

A Manager MUST manage at least 1 Department.
Reasoner enforces this constraint.

```
:Manager  
[minCard: 1 manages]
```

FunctionalProperty

Each Employee works for AT MOST ONE Organisation.
Reasoner enforces single-valued constraint.

```
:worksFor  
type: Functional
```

Auto-Inference

:Alice rdf:type :Doctor
Reasoner infers:
:Alice rdf:type :Employee
(Doctor subClassOf Employee — never stated!)

```
HermiT / Pellet  
reasoner infers this
```


CLOSED WORLD (Databases / Prolog)

If something is NOT in the database → assumed FALSE

SQL returns 0 rows → 'No such record exists'

⚠ DANGER: Bob's employer is not recorded → system concludes Bob has NO job.

In medicine: No allergy recorded → 'SAFE to prescribe penicillin'

This assumption is DANGEROUS in incomplete knowledge domains!

VS

OPEN WORLD (OWL Ontologies)

If something is NOT stated → it is UNKNOWN (not false)

OWL: Alice's allergy not recorded → 'allergy status UNKNOWN'

✅ SAFE: System says 'check allergy before prescribing'

In medicine: absence of data ≠ absence of condition

This assumption is SAFE in open, incomplete real-world knowledge!



CLASS ACTIVITY 5.1: Smart City Ontology Design

SCENARIO: A city government wants Traffic, Hospital, School, and Police systems to share data. Each system uses different vocabulary. Design the core ontology that unifies them.

Task 1

Define 5 CLASSES, each with 3 properties:

- 2 object properties
(links to another class)
- 1 data property
(links to a literal value)

Task 2

Write 4 OWL AXIOMS:

- 1 disjointWith
- 1 cardinality constraint
- 1 subClassOf restriction
- 1 transitive property

Task 3

INTEROPERABILITY CHALLENGE:

Traffic → 'Traffic Jam'
GPS → 'Congestion'
News → 'Road Blockage'

How does your ontology unify these 3 names?

Task 4

EXTENSION: One city has autonomous vehicles, another doesn't.

How do you extend the ontology WITHOUT breaking existing systems?



SOLUTION 5.1: Smart City Ontology Design

OWL AXIOMS

1. `:ActiveEmergency owl:disjointWith :ResolvedEmergency`
(cannot be both at once)
2. `:Emergency owl:cardinality 1 :occurredAt`
(every emergency has EXACTLY 1 location)
3. `:Paramedic subClassOf :Person AND [minCard 1 :hasQualification]`
(paramedic MUST have at least 1 qualification)
4. `:isPartOf rdf:type owl:TransitiveProperty`
(`Street → District → City` $\Rightarrow$ `Street isPartOf City`)

Interoperability — Unifying 3 Names

```
:TrafficJam owl:equivalentClass :RoadCongestion
:Congestion owl:equivalentClass :RoadCongestion
rdfs:label 'Road Blockage'@en
```

Now ALL 3 systems query the SAME concept!
Query for any of the 3 names → same results.

Classes with Properties

`:Location` — `hasAddress`, `isInDistrict` [`→:District`], `hasCoordinates`
`:Person` — `hasName`, `livesIn` [`→:Location`], `hasAge`
`:Vehicle` — `hasLicense`, `ownedBy` [`→:Person`], `isType`
`:Emergency` — `occurredAt` [`→:Location`], `severity`, `reportedBy` [`→:Person`]
`:PublicService` — `locatedAt` [`→:Location`], `operatedBy` [`→:Organization`], `isOpen`

Extension: Autonomous Vehicles

Add: `:AutonomousVehicle rdfs:subClassOf :Vehicle`

Cities without AVs simply never use this class.
Existing Vehicle queries still work — AV inherits everything.
Base ontology is UNCHANGED.

This is the Open World Assumption in practice!

OWL Modeling Concepts

Reasoning, consistency, and automated inference

Capstone

6.1 subClassOf vs rdf:type — Critical Distinction

Hour 6

`rdf:type` (instance-of)

```
:Fido rdf:type :Dog
```

- Fido IS an individual Dog.
- Fido is at the INSTANCE level.
- Fido CANNOT have subclasses.
- He is a specific animal, not a category.

ANALOGY: Like an object in Python

`fido = Dog()` ← Fido is an instance of Dog

✗ COMMON MISTAKE

```
:Fido rdfs:subClassOf :Dog
```

This makes Fido into a CLASS!
Now valid: `:Rex rdf:type :Fido` — Rex is a TYPE of Fido. NONSENSE!

`rdfs:subClassOf` (class hierarchy)

```
:Dog rdfs:subClassOf :Mammal
```

- The CLASS 'Dog' is a SUBSET of CLASS 'Mammal'.
- All instances of Dog are automatically Mammals.
- Dog is at the CLASS level — can have subclasses.

ANALOGY: Like a class in Python

`class Dog(Mammal): pass` ← Dog IS-A Mammal

6.2 Key OWL Relationships Reference

OWL Construct	Meaning	Example
owl:equivalentClass	Two classes = exact same set of members	:HeartAttack equivalentClass :MyocardialInfarction
owl:sameAs	Two individual names = same real entity	:MuhammadAli sameAs :CassiusClay
owl:disjointWith	Two classes share NO members — mutual exclusion	:Man disjointWith :Woman
owl:TransitiveProperty	$A \rightarrow B$ and $B \rightarrow C$ implies $A \rightarrow C$ (chain inference)	isAncestorOf, isPartOf, isLocatedIn
owl:FunctionalProperty	Each subject has AT MOST ONE value	hasBiologicalMother (max 1 per person)
owl:SymmetricProperty	If $A \rightarrow B$ then $B \rightarrow A$ is automatically true	isSiblingOf, isNear, isMarriedTo
owl:minCardinality n	Must have AT LEAST n values	Course minCard 1 hasEnrolledStudent
owl:cardinality n	Must have EXACTLY n values	Triangle cardinality 3 hasSide

1. CLASSIFICATION

You describe properties → reasoner places concept in correct hierarchy automatically.

```
Input: :Platypus lays-eggs=TRUE, warm-blooded=TRUE, has-  
fur=TRUE  
Reasoner output: :Platypus subClassOf :Mammal  
(INFERRED — never stated by you!)
```

2. CONSISTENCY CHECKING

Finds contradictions automatically — you cannot have an instance of two disjoint classes.

```
Input: :Alice rdf:type :Man. :Alice rdf:type :Woman.  
       :Man disjointWith :Woman.  
Output: INCONSISTENCY — Alice cannot be both.
```

3. INSTANCE CLASSIFICATION

Assigns individuals to the right class based on their properties.

```
Input: :Bob hasChild :Alice. :Bob rdf:type :Male.  
Output: :Bob rdf:type :Father  
(Father = Male AND hasChild≥1 — INFERRED!)
```

4. PROPERTY INFERENCE

Applies transitive, symmetric, and functional properties automatically.

```
Input: Heart isPartOf Body. Valve isPartOf Heart.  
       isPartOf is Transitive.  
Output: Valve isPartOf Body (NEVER stated!)
```



CLASS ACTIVITY 6.1: Ontology Consistency Debugging

SCENARIO: Find and fix 3 logical contradictions in this university ontology.

(1) **:Student**
disjointWith :Professor

Verdict: _____

(2) **:TeachingAssistant**
subClassOf :Student

Verdict: _____

(3) **:TeachingAssistant**
subClassOf :Professor

Verdict: _____

(4) **:Alice**
type :TeachingAssistant

Verdict: _____

(5) **:Course cardinality**
1 :hasProfessor

Verdict: _____

(6) **:CS101**
hasProfessor :ProfSmith

Verdict: _____

(7) **:CS101**
hasProfessor :ProfJones

Verdict: _____

(8) **:FullProfessor**
subClassOf :Professor

Verdict: _____

(9) **:FullProfessor**
subClassOf :Man

Verdict: _____

(10) **:ProfSmith**
type :FullProfessor

Verdict: _____

(11) **:ProfSmith**
type :Woman

Verdict: _____

For each: CORRECT / REDUNDANT / CONTRADICTION — explain why. Then FIX the 3 contradictions!

✓ SOLUTION 6.1 (Part 1): Ontology Consistency Debugging

CORRECT

(1) `:Student disjointWith :Professor`

Needed to define conflict

CORRECT

(2) `TA subClassOf :Student`

TAs are students

CONTRADICTION!

(3) `TA subClassOf :Professor`

Combined with (1)(2)(4) → Alice is BOTH → violates `disjointWith`

CORRECT

(4) `:Alice type :TeachingAssistant`

Alice is a TA

CONTRADICTION!

(5) `:Course cardinality 1 :hasProfessor`

CS101 has Smith AND Jones — violates exactly-1

CONTRADICTION!

(6–7) `CS101 has both Smith & Jones`

Facts 6+7 together violate cardinality from fact 5

CORRECT

(8) `:FullProfessor
subClassOf :Professor`

Structural hierarchy

CONTRADICTION!

(9) `:FullProfessor subClassOf :Man`

ProfSmith (FullProf) → :Man, but Smith is :Woman →
Man `disjointWith` Woman

CONTRADICTION!

(10–11) `ProfSmith FullProf + Woman`

Via (9): FullProf → Man + Woman → violates
`disjointWith`



SOLUTION 6.1 (Part 2): Fixes & Derived Facts

Contradiction 1: TA cannot be both Student & Professor

FIX: REMOVE the disjointWith between Student and Professor (too strict — TAs exist).
Create :AcademicRole as superclass. Both are sub-roles.
TA subClassOf :Student AND subClassOf :Instructor (new class).

Contradiction 2: CS101 has 2 professors but cardinality=1

FIX: CHANGE cardinality 1 to minCardinality 1 (at least 1 professor).
OR model co-teaching with a :TeachingTeam class.
Both preserve the reality of how universities actually work.

Contradiction 3: FullProfessor subClassOf Man

FIX: DELETE fact (9) entirely. FullProfessor subClassOf Professor is sufficient.
Gender is irrelevant to academic rank. This fact encodes historical bias in formal ontology.

BONUS: Fact (9) is bad ontology design — it logically forbids female full professors and encodes historical bias into formal structure. Good ontology models WHAT a thing IS (a rank), not irrelevant attributes (gender).



CLASS ACTIVITY 6.2: Real-World Case: Healthcare OWL Ontology — Automated Diagnosis Reasoning

REAL SCENARIO: HL7 FHIR (Fast Healthcare Interoperability Resources) uses OWL ontologies so hospitals worldwide share patient data. A doctor in Lahore can treat a patient whose records were created in London — because both systems use the SAME ontology. You design part of it today.

Task 1

Write OWL axioms for:

- Drug subClassOf Medication
- Antibiotic subClassOf Drug
- Penicillin rdf:type Antibiotic
- Drug owl:hasProperty dosage
- dosage owl:range xsd:float

Draw the class diagram.

Task 2

CONSTRAINT DESIGN:

Patient MUST have:

- bloodType $\in \{A,B,AB,O\}$
- age range: 0–150
- at most ONE primary physician

Write each as OWL restriction
(owl:allValuesFrom,
owl:maxCardinality, owl:oneOf).

Task 3

AUTOMATED REASONING:

Given:

- Ali rdf:type Patient
- Ali has symptom: fever, cough, fatigue
- COVID-19 owl:equivalentClass (symptom SOME {fever, cough, fatigue})

What should a reasoner INFER?
Show the logical chain.

Task 4

CONSISTENCY CHECK:

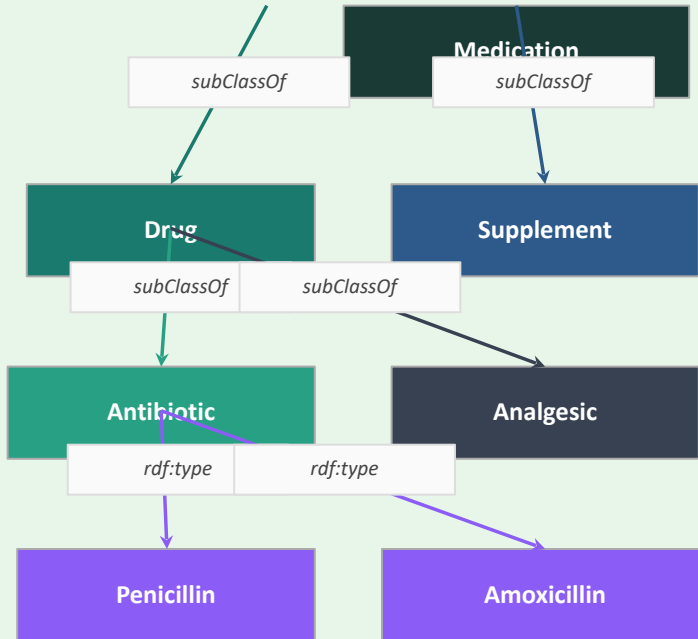
Find the error:

- Drug owl:disjointWith Food
- VitaminC rdf:type Drug
- VitaminC rdf:type Food

What happens when a reasoner runs? How do you fix it?
Name a real drug that IS borderline (food/drug).

✓ SOLUTION 6.2: Healthcare OWL Ontology — Class Diagram & Reasoning

OWL Class Hierarchy



✗ Consistency Error (Task 4)

Drug owl:disjointWith Food
VitaminC rdf:type Drug AND rdf:type Food
→ CONTRADICTION → reasoner flags UNSATISFIABLE

FIX: Remove disjoint OR create subclass:
NutraceuticalDrug subClassOf Drug
VitaminC rdf:type NutraceuticalDrug
Real example: Fish Oil, Melatonin, Probiotics

OWL Constraints

```
Patient owl:hasProperty bloodType
bloodType owl:oneOf {A, B, AB, O}
age owl:minInclusive 0, owl:maxInclusive 150
hasPrimaryPhysician owl:maxCardinality 1
```

⚡ Automated Reasoning Chain (Task 3)

GIVEN:

```
Ali rdf:type Patient
Ali hasSym {fever, cough, fatigue}
```

OWL RULE:

```
COVID-19 ≡ Patient AND hasSym SOME {fever, cough, fatigue}
```

INFERRED:

```
Ali rdf:type COVID-19 ← reasoner derives this!
```

